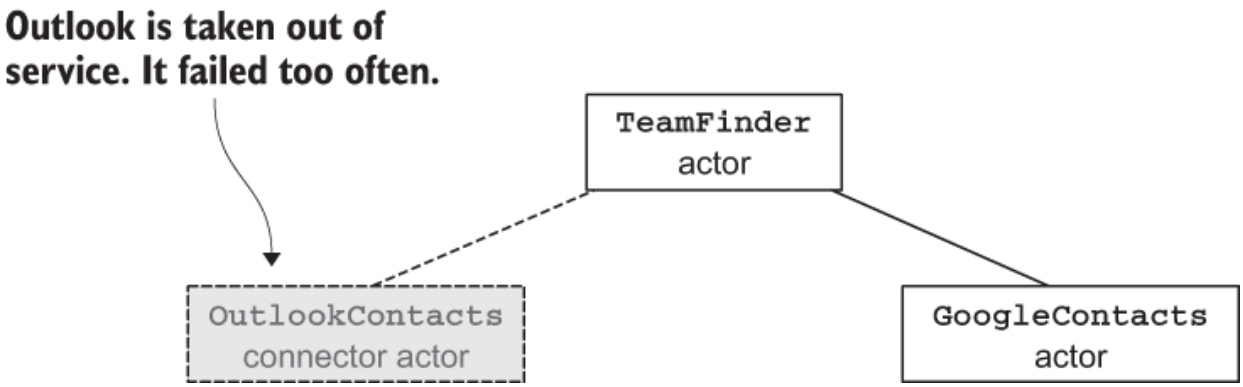


Figure 1.19 `TeamFinder` supervising contacts actors

Actors: Decoupled on three levels

Another way to look at actors is how they're decoupled for scaling on three levels: space/location, time, and interface. Actors can run simultaneously if there are enough CPUs or run one after the other if not. Actors can be co-located or far apart; and in a failure scenario, actors may receive messages they can't handle.
Decoupling on these three levels is important because this is exactly the flexibility required for scaling:

- *Space*—An actor gives no guarantee and has no expectations about where another actor is located.
- *Time*—An actor gives no guarantee and has no expectations about when its work will be done.
- *Interface*—An actor has no defined interface. Nothing is shared between actors; actors never point to or use a shared piece of information that changes in place. Information is passed in messages.

Coupling components in location, time, and interface is the biggest impediment to building applications that can recover from failure and scale according to demand. A system built out of components that are coupled on all three can only exist on one runtime and will fail completely if one of its components fails.

Now that you've looked at the operations an actor can perform, let's look at how Akka supports actors and what's required to make them